Terraform Labs

None

None

None

Table of contents

1. Installation 3
   1.1 Installer Terraform 3
2. Premiers Pas 4
   2.1 Premiers Pas 4
3. Graph 7
   3.1 Graph 7
4. L'Inventaire dynamique 9
   4.1 Lab3: L'inventaire dynamique 9
   4.2 Lab3: L'inventaire dynamique 10
5. Les PlayBooks 11
   5.1 Lab4: Les PlayBooks 11
   5.2 Lab4: L'Inventaire 12
6. Jinja2 15
   6.1 Lab5: Jinja2 15
   6.2 Lab5: Jinja2 16
7. Roles 17
   7.1 Lab6 Les rôles 17
   7.2 Lab6 Les rôles 18
8. Vault 20
   8.1 Lab7 Vault 20
   8.2 Lab7 Vault 21
9. Network 22
   9.1 Lab8 Network 22
   9.2 Lab8 Network 23
   9.3 Lab9 solution 24

1. Installation

1.1 Installer Terraform

1.1.1 Installer

Télécharger le binaire **terraform** à partir de https://terraform.io/downloads.html

Solution

```
curl -O https://releases.hashicorp.com/terraform/1.0.9/terraform_1.0.9_linux_amd64.zip
```

Extraitre le binaire et l'installer sur **/usr/local/bin**

Solution

```
unzip terraform_1.0.9_linux_amd64.zip
sudo install terraform /usr/local/bin
```

1.1.2 Verifier

Vérifier la version de terraform

Solution

```
terraform --version
```

2. Premiers Pas

2.1 Premiers Pas

2.1.1 Initialisation

Créer un répertoire **www** Créer le fichier **main.tf** pour initialiser l'environnement **terraform** avec le provider **aws**

Solution

```
mkdir www && cd www
vi main.tf
```

```
terraform {
    required_version = ">= 0.12"
}
provider "aws" {
    region = "eu-central-1"
}
```

Initialiser l'environnement de travail en utilisant la commande **terraform init**

Solution

```
terraform init
```

- La commande init a pour conséquence de télécharger les providers et les modules nécessaires au fonctionnement de votre configuration (seul le provider AWS sera téléchargé dans notre cas).

2.1.2 Création d'une infrastructure

Créer une instance ec2 avec les caractéristiques suivantes :

- ami : ami-0f7cd40eac2214b37
- instance_type : t2.micro

Solution

```
vi main.tf
```

```
resource "aws_instance" "www" {
  ami           = "ami-0f7cd40eac2214b37"
  instance_type = "t2.micro"
}
```

Utilisez la commande **terraform plan** pour analyser le plan d'exécution de votre configuration. Cette commande ne provoque aucune modification dans AWS

Solution

```
terraform plan
```

Une fois que le résultat de la commande précédente vous convient, vous pouvez créer votre infrastructure avec la commande **terraform apply**

Solution

```
terraform apply
```

- Vous pourrez vérifier la création de l'infrastructure en utilisant la console AWS (un lien est sur le bureau de votre VM)

2.1.3 Modification d'une infrastructure

Ajout d'un tag

Proposez une solution vous permettant de nommer votre instance afin de la différencier de celles des autres participants à la formation Ajouter l'attribut **tags** à votre instance ec2 :

- tags : { Name = Votre-Nom }

Solution

```
vi main.tf
```

```
resource "aws_instance" "www" {
  ami           = "ami-0f7cd40eac2214b37"
  instance_type = "t2.micro"
  tags = {
      Name = "Samir"
  }
}
```

Ajout d'une clé ssh

Proposez et mettez en place une solution pour permettre la connexion à vote instance via SSH en utilisant la ressource aws_key_pair mise à disposition par le provider AWS.

Solution

```
ssh-keygen
```

```
Generating public/private rsa key pair.
Enter file in which to save the key (/home/retengr/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/retengr/.ssh/id_rsa
Your public key has been saved in /home/retengr/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:NplnvBx4xSalwYr4uNN6DZulHo/9MCXQEzPH8Yv9WgI retengr@ip-10-0-9-110
The key's randomart image is:
+---[RSA 3072]----+
|        +o+..    |
|       . =o*     |
|     ...o.o =    |
|    . ...* * .   |
|     o  S E o    |
|    . o..O + .   |
|     o.Bo o . o  |
|    o *=.o   +   |
|    .=o o.. .    |
+----[SHA256]-----+
```

Une fois les clés générées, il faut créer la ressource de type aws_key_pair. La création de cette ressource nécessite de copier votre clé publique. Il ne reste ensuite plus qu'à associer cette ressource à l'instance à créer. Attention, la clé que vous allez déclarer dans AWS doit avoir un nom unique, vous prendrez soin de mettre votre prénom dans le nom de cette dernière afin d'éviter les conflits avec les clés créées par les autres participants.

Solution

```
resource "aws_key_pair" "rtg-connect-samir" {
  key_name = "rtg-key-samir"
  public_key= "AAAAB3NzaC1yc2EAAAADAQABAAABAQC1hm3hwsrmSpRcLqfndGtdafMWZCxK0Jf9QJ6FNXIojtDCH9jW2bRSd94YHyYzN+L58kcENhicv8pDZdBesYvTO/atj6VBkKWuwlYdXw3kyTZq+2Lj56/
Dhds/J6gRWRmkzQJpGkGgdlPeK94ql+2/wwZKzJzV6DAdmaOWzTcBoJu1srVMdRnrMekFbWEiLn98XEF6Y6oBtolkJ6p5EId8MadIksh/zu/
CzvZWbuaUIPQHvpbIAl0bgallxz46nZdFNc7pxXAbxYp5pVkvB7K6oqosBy25PCZwfq7q/TToT4HMFMNbegNYFFN8fH4X9YUdmeF18ax/eSxrNP+HNRNx"
}
```

Mise en place de Security Group

Un security Group (SG) est une ressource AWS qui permet de contrôler les flux réseau qui pourront entrer dans une instance EC2 (INGRESS) ou sortir de cette même instance (EGRESS). Il s'agit d'un mécanisme de sécurité important qu'il convient de mettre en place pour chacun des composants déployés. Utilisez la ressource aws_security_group afin de définir les règles de filtrages suivantes :

- Ingress : Port 22 ouvert (ssh)
- Egress : Totalement ouvert

Solution

```
vi main.tf
```

```
resource "aws_security_group" "only-ssh" {
  name        = "onlyssh"
  description = "Only ssh"
    ingress {
      description = "ssh"
      from_port   = 22
      to_port     = 22
      protocol    = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
    egress {
        from_port   = 0
        to_port     = 0
        protocol    = "-1"
        cidr_blocks = ["0.0.0.0/0"]
    }
  tags = {
    Name = "Only ssh"
  }
}
```

Ajouter la clé **ssh** et la clé de sécurité dans la description de l'instance **ec2**

Solution

```
vi main.tf
```

```
resource "aws_instance" "www" {
  ami           = "ami-0f7cd40eac2214b37"
  instance_type = "t2.micro"
  security_groups = [ aws_security_group.only-ssh.name ]
  key_name = "rtg-key"
  tags = {
      Name = "Samir"
  }
}
```

3. Graph

3.1 Graph

3.1.1 Terraform Graph

La commande terraform grapp

Avant d'appliquer le plan pour créer réellement l'infrastructure, on peut utiliser la commande Terraform graph pour générer une représentation visuelle de la configuration ou du plan d'exécution.

```
terrform graphp --help
```

```
Usage: terraform [global options] graph [options]

  Outputs the visual execution graph of Terraform resources according to
  either the current configuration or an execution plan.

  The graph is outputted in DOT format. The typical program that can
  read this format is GraphViz, but many web services are also available
  to read this format.

  The -type flag can be used to control the type of graph shown. Terraform
  creates different graphs for different operations. See the options below
  for the list of types supported. The default type is "plan" if a
  configuration is given, and "apply" if a plan file is passed as an
  argument.

Options:

  -plan=tfplan     Render graph using the specified plan file instead of the
                   configuration in the current directory.

  -draw-cycles     Highlight any cycles in the graph with colored edges.
                   This helps when diagnosing cycle errors.

  -type=plan       Type of graph to output. Can be: plan, plan-destroy, apply,
                   validate, input, refresh.

  -module-depth=n  (deprecated) In prior versions of Terraform, specified the
                   depth of modules to show in the output.
```

Exemple d'utilisation

voici un fichier avec quelques ressources

```
cat maint.tf
```

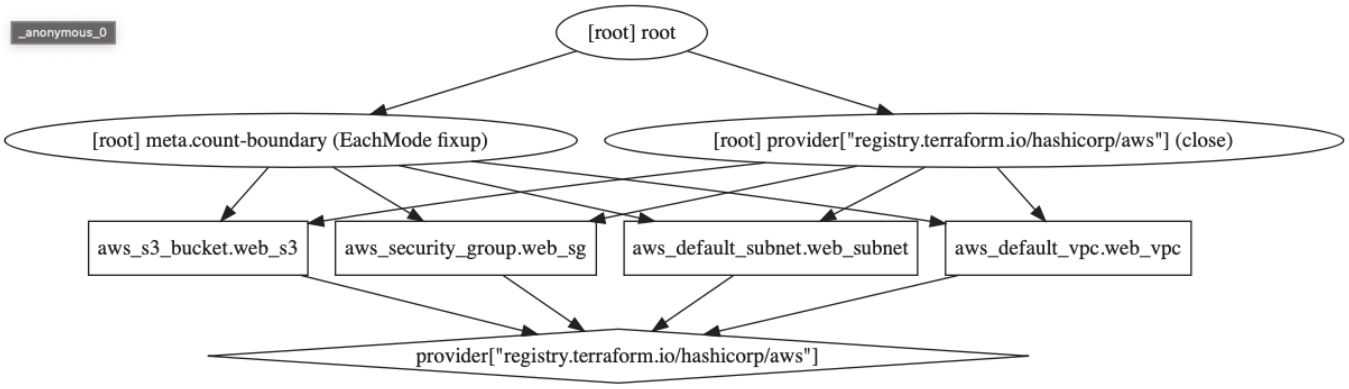
```
provider "aws" {
  region  = "eu-west-3"
}

resource "aws_s3_bucket" "web_s3" {
  bucket = "s3-20201103"
}

resource "aws_default_vpc" "web_vpc" {
  tags = {
    Name = "Default VPC"
  }
}

resource "aws_default_subnet" "web_subnet" {
  availability_zone = "eu-west-3a"
}

resource "aws_security_group" "web_sg" {
  name        = "webserver"
  description = "Acces HTTP"
}
```

Exécution de la commande **terraform graph** Elle fournit une sortie au format **dot**

```
terraform graph
```

```
digraph {
    compound = "true"
    newrank = "true"
    subgraph "root" {
        "[root] aws_default_subnet.web_subnet (expand)" [label = "aws_default_subnet.web_subnet", shape = "box"]
        "[root] aws_default_vpc.web_vpc (expand)" [label = "aws_default_vpc.web_vpc", shape = "box"]
        "[root] aws_s3_bucket.web_s3 (expand)" [label = "aws_s3_bucket.web_s3", shape = "box"]
        "[root] aws_security_group.web_sg (expand)" [label = "aws_security_group.web_sg", shape = "box"]
        "[root] provider[\"registry.terraform.io/hashicorp/aws\"]" [label = "provider[\"registry.terraform.io/hashicorp/aws\"]", shape = "diamond"]
        "[root] aws_default_subnet.web_subnet (expand)" -> "[root] provider[\"registry.terraform.io/hashicorp/aws\"]"
        "[root] aws_default_vpc.web_vpc (expand)" -> "[root] provider[\"registry.terraform.io/hashicorp/aws\"]"
        "[root] aws_s3_bucket.web_s3 (expand)" -> "[root] provider[\"registry.terraform.io/hashicorp/aws\"]"
        "[root] aws_security_group.web_sg (expand)" -> "[root] provider[\"registry.terraform.io/hashicorp/aws\"]"
        "[root] meta.count-boundary (EachMode fixup)" -> "[root] aws_default_subnet.web_subnet (expand)"
        "[root] meta.count-boundary (EachMode fixup)" -> "[root] aws_default_vpc.web_vpc (expand)"
        "[root] meta.count-boundary (EachMode fixup)" -> "[root] aws_s3_bucket.web_s3 (expand)"
        "[root] meta.count-boundary (EachMode fixup)" -> "[root] aws_security_group.web_sg (expand)"
        "[root] provider[\"registry.terraform.io/hashicorp/aws\"] (close)" -> "[root] aws_default_subnet.web_subnet (expand)"
        "[root] provider[\"registry.terraform.io/hashicorp/aws\"] (close)" -> "[root] aws_default_vpc.web_vpc (expand)"
        "[root] provider[\"registry.terraform.io/hashicorp/aws\"] (close)" -> "[root] aws_s3_bucket.web_s3 (expand)"
        "[root] provider[\"registry.terraform.io/hashicorp/aws\"] (close)" -> "[root] aws_security_group.web_sg (expand)"
        "[root] root" -> "[root] meta.count-boundary (EachMode fixup)"
        "[root] root" -> "[root] provider[\"registry.terraform.io/hashicorp/aws\"] (close)"
    }
}
```

faites un Copy/pAste sur le site webGraphViz on obtien ce type de graph



Graph en mode Local

Installer Graphiz

```
sudo apt install graphiz
```

Utiliser la commande **dot** pour générer un graphique

```
terraform graph | dot -T svg > graph.svg
```

Blast Radius

```
pip install blastraduis
```

Démarrer le serveur

```
blast-radius --serve .
```

4. L'Inventaire dynamique

4.1 Lab3: L'inventaire dynamique

4.1.1 Créer un inventaire dynamique pour virtualbox

4.1.2 Afficher l'inventaire

4.1.3 Tester les cibles

4.2 Lab3: L'inventaire dynamique

4.2.1 Créer un inventaire dynamique pour virtualbox

```
vi vbox.yml
```

```
plugin: virtualbox
running_only: yes
groups:
  web: "'web' in (inventory_hostname)"
  ha: "'ha' in (inventory_hostname)"
```

4.2.2 Afficher l'inventaire

```
ansible-inventory -i vbox.yml --graph
```

4.2.3 Tester les cibles

```
ansible all -i vbox.yml -m ping
```

4.2.4 Créer les VMs

```
VBoxManage import centos7orig.ova --dry-run
```

```
VBoxManage import centos77last.ova  --vsys 0 --vmname  ha --memory 1024 --unit 4 --ignore --unit 5 --ignore --unit 8 --ignore --unit 9 --ignore --unit 10 --
ignore --unit 11 --ignore --unit 12 --disk /Users/samir/virtualbox/kube/kube-disk001.vmdk
```

```
VBoxManage import centos77last.ova  --vsys 0 --vmname n1 --memory 1024 --unit 4 --ignore --unit 5 --ignore --unit 8 --ignore --unit 9 --ignore --unit 10 --
ignore --unit 11 --ignore --unit 12 --disk /Users/samir/virtualbox/n1/n1-disk001.vmdk
```

```
VBoxManage import centos77last.ova  --vsys 0 --vmname n2 --memory 1024 --unit 4 --ignore --unit 5 --ignore --unit 8 --ignore --unit 9 --ignore --unit 10 --
ignore --unit 11 --ignore --unit 12 --disk /Users/samir/virtualbox/n2-disk001.vmdk
```

```
VBoxManage import centos77last.ova  --vsys 0 --vmname n3 --memory 1024 --unit 4 --ignore --unit 5 --ignore --unit 8 --ignore --unit 9 --ignore --unit 10 --
ignore --unit 11 --ignore --unit 12 --disk /Users/samir/virtualbox/n3-disk001.vmdk
```

5. Les PlayBooks

5.1 Lab4: Les PlayBooks

5.1.1 Installer les deux serveurs web

5.1.2 Copier des fichiers distincts

5.1.3 Démarrer les services

5.1.4 Installer le haproxy

5.1.5 Copier le fichier de config haproxy.cfg

5.1.6 Ajouter un handler de restart du service haproxy lors de la copie d'un nouveau fichier de config

5.2 Lab4: L'Inventaire

5.2.1 Installer les deux serveurs web

```
vi web.yml
```

```
---
- name: "Installation de http sur les serveurs WEB"
  hosts: web
  become: yes
  tasks:
  - name: Install http
    yum:
      name: httpd
      state: present
```

5.2.2 Copier des fichiers distincts

```
vi web.yml
```

```
- name: "copie index host112"
  hosts: host112
  become: yes
  tasks:
  - name: Copie du fichier index.html
    copy:
      src: index1.html
      dest: /var/www/html/index.html
      owner: apache
      group: apache
      mode: 0644
- name: "copie index host113"
  hosts: host113
  become: yes
  tasks:
  - name: Copie du fichier index.html
    copy:
      src: index2.html
      dest: /var/www/html/index.html
      owner: apache
      group: apache
      mode: 0644
```

5.2.3 Démarrer les services web

```
vi web.yml
```

```
- name: "Start des serveurs WEB"
  hosts: web
  become: yes
  tasks:
  - name: Ensure the http service is running
    service:
      name: httpd
      enabled: yes
      state: started
```

5.2.4 Installer le haproxy

```
vi web.yml
```

```
---
- name: installation haproxy
  hosts: host111
  become: yes
  tasks:
  - name: Install haproxy
    yum:
      name: haproxy
      state: present
```

5.2.5 Copier le fichier de config haproxy.cfg

```
vi web.yml
```

```
---
- name: installation haproxy
  hosts: host111
  become: yes
  tasks:
  - name: Copie du fichier de configuration
    copy:
      src: "{{ config }}"
      dest: /etc/haproxy/haproxy.cfg
      owner: haproxy
      group: haproxy
      mode: 0644
```

5.2.6 Ajouter un handler de restart du service haproxy lors de la copie d'un nouveau fichier de config

```
vi web.yml
```

```
---
- name: installation haproxy
  hosts: host111
  become: yes
  handlers:
  - name: Restart haproxy
    service:
      name: haproxy
      state: restarted
  tasks:
  - name: Copie du fichier de configuration
    copy:
      src: "{{ config }}"
      dest: /etc/haproxy/haproxy.cfg
      owner: haproxy
      group: haproxy
      mode: 0644
    notify: Restart haproxy
```

5.2.7 Le fichier complet

```
vi web.yml
```

```
---
- name: installation haproxy
  hosts: host111
  become: yes
  handlers:
  - name: Restart haproxy
    service:
      name: haproxy
      state: restarted
  tasks:
  - name: Install haproxy
    yum:
      name: haproxy
      state: present
  - name: Copie du fichier de configuration
    copy:
      src: "{{ config }}"
      dest: /etc/haproxy/haproxy.cfg
      owner: haproxy
      group: haproxy
      mode: 0644
    notify: Restart haproxy
  - name: Ensure the http service is running
    service:
      name: haproxy
      enabled: yes
      state: restarted
- name: "Installation de http sur les serveurs WEB"
  hosts: web
  become: yes
  tasks:
  - name: Install http
    yum:
      name: httpd
      state: present
  - name: Ensure the http service is running
    service:
      name: httpd
      enabled: yes
```

```
      state: started
- name: "firewall port 80"
  hosts: all
  become: yes
  tasks:
  - firewalld:
      port: 80/tcp
      permanent: true
      immediate: true
      zone: public
      state: enabled
- name: "copie index host112"
  hosts: host112
  become: yes
  tasks:
  - name: Copie du fichier index.html
    copy:
      src: index1.html
      dest: /var/www/html/index.html
      owner: apache
      group: apache
      mode: 0644
- name: "copie index host113"
  hosts: host113
  become: yes
  tasks:
  - name: Copie du fichier index.html
    copy:
      src: index2.html
      dest: /var/www/html/index.html
      owner: apache
      group: apache
      mode: 0644
```

5.2.8 Le fichier de config HAproxy

```
vi haproxy.cfg
```

```
frontend http_frontend
bind *:80
        default_backend http_backend
backend http_backend
        mode http
        balance roundrobin
        server server1 host112:80 check
        server server2 host113:80 check
```

5.2.9 Le fichier inventaire

```
vi inventory
```

```
[ha]
host111 config=haproxy.cfg [web]
host112
host113
[web:vars]
index=index.html
```

6. Jinja2

6.1 Lab5: Jinja2

6.1.1 Créer un user admin sur le serveur haproxy avec un fichier bashrc spécifique pour modifier le prompt = user@hostname

6.1.2 Utiliser un template pour créer un fichier index.html 'echo hostname'

6.1.3 Afficher pour les serveurs la taille RAM et calculer un seuil de saturation de 70% RAM

6.2 Lab5: Jinja2

6.2.1 Créer un user admin sur le serveur haproxy avec un fichier bashrc spécifique pour modifier le prompt = user@hostname

```
vi bashrc.j2
```

```
export PS1="{{ inventory_hostname }}:{{ user_name }} $ "
```

6.2.2 Utiliser un template pour créer un fichier index.html 'echo hostname'

```
vi host.html.j2
```

```
<html>
<head> <title>{{inventory_hostname}}</title> </head>
<body>
<p>Nom du hôte : {{inventory_hostname}}</p>
<p>Ci-dessous la liste de ces adresses :</p>
<ul>
{% for ip in ansible_all_ipv4_addresses %}
<li>{{ip}}</li>
{% endfor %}
</ul>
</body>
</html>
```

6.2.3 Afficher pour les serveurs la taille RAM et calculer un seuil de saturation de 70% RAM

```
vi playbook.yml
```

```
---
- name: "Generate html file for each host"
  hosts: web
  gather_facts: yes
  become: yes
  tasks:
    - name: "html file generation"
      template:
        src: "host.html.j2"
        dest: "/var/www/html/index.html"
    - name: Checking Memory
      debug:
        msg: "{% if ansible_memfree_mb < (ansible_memtotal_mb * 0.7)  %} mémoire suffisante {% else %} Rmémoire insuffisante {% endif %}"
- name: "Create User and profile"
  hosts: ha
  gather_facts: yes
  become: yes
  vars:
    user_name: admin
    user_state: present
  tasks:
    - name: Create user on remote host
      user:
        name: '{{user_name}}'
        state: '{{user_state}}'
        remove: yes
        shell: /bin/bash
    - name: Add bashrc to include host and user
      template:
        dest: '~{{user_name}}/.bashrc'
        src: bashrc.j2
```

7. Roles

7.1 Lab6 Les rôles

7.1.1 Créer un rôle user_create et créer un user avec ce rôle sur le serveur haproxy avec un fichier bashrc spécifique pour modifier le prompt

7.1.2 Créer la variable {{user}} différents niveau d'initialisation

1/ inventaire 2/ playbook 3/ répertoire 4/ rôle

7.1.3 Installer un rôle docker avec Ansible-Galaxy

7.1.4 Intaller docker sur le serveur ha

7.2 Lab6 Les rôles

7.2.1 Créer un rôle user_create et créer un user avec ce rôle sur le serveur haproxy avec un fichier bashrc spécifique pour modifier le prompt

Initialiser le Rôle

```
mkdir ~/roles
cd roles
ansible-galaxy init user_create
```

```
- Role user_create was created successfully
```

Modifier les fichiers du Rôle

```
vi roles/user_create/vars/main.yml
```

```
---
# vars file for user_create
user_name: admin
user_state: present
```

```
vi roles/user_create/templates/bashrc.j2
```

```
export PS1='{{inventory_hostname}}:{{user_name}} $ '
```

Modifier le fichier tsk du rôle

```
vi roles/user_create/tasks/main.yml
```

```
---
# tasks file for user_create
- name: Create user on remote host
  become: yes
  user:
    name: '{{user_name}}'
    state: '{{user_state}}'
    shell: /bin/bash
    password: '{{user_password}}'
    append: yes
- name: Add bashrc to include host and user
  become: yes
  template:
    dest: '~{{user_name}}/.bashrc'
    src: templates/bashrc.j2
```

Créer le playbook qui utilise le Rôle

```
vi playbook.yml
```

```
---
- hosts: web
  vars:
    user_password: "{{ 'my_passwd' |password_hash('sha512') }}"
  roles:
    - { role: user_create , become: yes }
```

7.2.2 Créer la variable "user" à différents niveau d'initialisation

1/ inventaire

2/ playbook

3/ répertoire

4/ rôle

7.2.3 Installer un rôle docker avec Ansible-Galaxy

```
ansible-galaxy install geerlingguy.docker
```

```
- downloading role 'docker', owned by geerlingguy
- downloading role from https://github.com/geerlingguy/ansible-role-docker/archive/3.1.2.tar.gz
- extracting geerlingguy.docker to /home/centos/.ansible/roles/geerlingguy.docker
- geerlingguy.docker (3.1.2) was installed successfully
```

7.2.4 Intaller docker sur le serveur ha avec le rôle docker

```
vi playbook.yml
```

```
---
- hosts: web
  vars:
    user_name: user_play
  roles:
    - { role: user_create , become: yes }
- hosts: ha
  roles:
    - { role: geerlingguy.docker , become: yes }
```

8. Vault

8.1 Lab7 Vault

8.1.1 Créer un Vault Password dans group_vars

8.1.2 Créer une variable passwd dans group_vars

8.1.3 Créer le mot de passe encrypté

8.1.4 Créer un template Jinja2 qui utilise le VaultPasswd

8.1.5 Créer et exécuter un playbook qui utilise le template Jinja2

8.2 Lab7 Vault

8.2.1 Créer un Vault Password dans group_vars

```
vi group_vars/all/vault
```

```
vault_password: thisIsnotAgoodPassword
```

8.2.2 Créer une variable passwd dans group_vars

```
vi group_vars/all/vars
```

```
password: '{{vault_password}}'
```

8.2.3 Créer le mot de passe encrypté

```
ansible-vault encrypt group_vars/all/vault
```

```
New Vault password:
Confirm New Vault password:
```

8.2.4 Créer un template Jinja2 qui utilise le VaultPasswd

```
vi password.j2
```

```
The password is {{password}}
```

8.2.5 Créer un playbook qui utilise le template Jinja2

```
vi pass.yml
```

```
---
- hosts: all
  tasks:
  - name: embed the secure password in a file
    template:
      src: password.j2
      dest: /tmp/password
      mode: 0600
```

8.2.6 Créer un playbook qui utilise le template Jinja2

```
ansible-playbook  pass.yml --ask-vault
```

9. Network

9.1 Lab8 Network

9.1.1 Créer une interface supplémentaire

1. utiliser le module nmcli
2. Ajouter une interface avec une ip +3 par rapport à la première IP

9.2 Lab8 Network

9.2.1 Créer une interface supplémentaire

1. utiliser le module nmcli
2. Ajouter une interface avec une ip +3 par rapport à la première IP

```
---
- name: config ip
  hosts: host111
  become: yes
  handlers:
  - name: Restart Network
    service:
      name: network
      state: restarted

  tasks:
    - set_fact: ip="{{ ansible_default_ipv4.address }}"
    - set_fact: hostint="{{ ip|ipaddr('int')|int()+3 }}"
    - debug:
        msg: hostip "{{ hostint }}"
    - set_fact: hostip="{{ hostint|ipv4('address') }}"
    - debug:
        msg: hostip "{{ hostip }}"
    - name: config ip
      nmcli:
        conn_name: ens224
        ifname: ens224
        type: ethernet
        ip4: "{{ hostip }}/24"
        state: present
        autoconnect: yes
      notify: Restart Network
```

9.3 Lab9 solution

Ennoncé

Créer un containe basé sur l'image nginx

Solution

```
#include <iostream>      int main(void) {       std::cout << "Hello world!" << std::endl;       return 0;     }
```

Créer un containe basé sur l'image nginx

Solution

```
Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla et euismod
nulla. Curabitur feugiat, tortor non consequat finibus, justo purus auctor
massa, nec semper lorem quam in massa.
```

Solution

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Nulla et euismod nulla. Curabitur feugiat, tortor non consequat finibus, justo purus auctor massa, nec semper lorem quam in massa.

```
def bubble_sort(items):
    for i in range(len(items)):
        for j in range(len(items) - 1 - i):
            if items[j] > items[j + 1]:
                items[j], items[j + 1] = items[j + 1], items[j]
```

Nunc eu odio eleifend, blandit leo a, volutpat sapien. Phasellus posuere in sem ut cursus. Nullam sit amet tincidunt ipsum, sit amet elementum turpis. Etiam ipsum quam, mattis in purus vitae, lacinia fermentum enim.